Paper 05 - Oloid Path Carrier

CQE-paper-05
CQECMPLX Formal Paper Series
Review-facing PDF built 2026-06-12

Construction Basis

Use curved/rolling carriers to show that transport need not be straight-line to preserve continuity.

This PDF is the clean paper artifact. Source extracts, kernels, receipts, tool notes, workbook material, and package bindings are used as construction evidence; they are not treated as separate review-facing papers.

Evidence Inputs

- promoted formal paper: production\formal-papers\CQE-paper-05\FORMAL_PAPER.md
- paper kernel manifest: production\paper-kernels\papers\CQE-paper-05\paper_kernel_manifest.json
- verifier: production\formal-papers\CQE-paper-05\verify_oloid_path_carrier.py
- receipt: production\formal-papers\CQE-paper-05\oloid_path_carrier_receipt.json (Rolling path carrier: pass)
- body: production\papers\CQE-paper-05\PAPER-BODY.md
- source: production\papers\CQE-paper-05\SOURCE.md
- formal: production\papers\CQE-paper-05\01-CQE-formal\FORMAL.md
- tool: production\papers\CQE-paper-05\02-CQE-tool\TOOL.md
- tool_runner: production\papers\CQE-paper-05\02-CQE-tool\run.py
- workbook: production\papers\CQE-paper-05\03-CQE-workbook\WORKBOOK.md
- recovered_output: production\lib-forge\recovered\papers_output\CQE-paper-05.md

Paper 05 - Oloid Path Carrier

Abstract

Paper 05 formalizes the path carrier that receives repaired boundary constraints from Paper 04. Its polished claim is structural: transport does not need to be represented as a straight-line jump in order to preserve continuity. A rolling-state carrier can preserve adjacency, order, and receipt state across a curved or cyclic path.

This paper does not claim that the Oloid model is already a complete Rule 30 predictor. Existing tests mark that mapping as speculative. What is verified here is the carrier property: a finite rolling state advances by legal adjacent steps, records a head/tail dyad, carries repaired constraints, and rejects invalid discontinuities.

Proof/Exposure Hierarchy

The proof-carrying content of this paper is the mathematics: the definitions, lemmas, constructions, examples, and receipts that establish the claimed result. Paper 00, hand routes, analog tools, workbook language, and obligation ledgers are supplemental validation and exposure layers. They exist to make the math inspectable, reproducible, and accessible without requiring a particular software stack. The hand route is not the purpose of the paper; it is a way to expose the same state transitions with ordinary marks, tokens, lines, or any equivalent physical substitute.

Role in the Suite

Paper 04 converts failed joins into typed constraints. Paper 05 provides a path carrier that can move those constraints forward without requiring a straight line between source and target.

The dependency chain is:

```
Paper 01: LCR local state
Paper 02: correction residue
Paper 03: coordinate surface
Paper 04: typed boundary constraint
Paper 05: rolling path carrier for that constraint
```

Definitions

A **rolling carrier state** is a triple

```
q = (sheet, phase, parity)
```

with

```
sheet in {0,1}
phase in {0,1,2,3}
parity in {0,1}
```

Given a bit `b in {0,1}`, define the rolling step:

```
roll(q,b) = ((sheet+1) mod 2, (phase+1) mod 4, parity xor b).
```

The **head/tail dyad** is

```
head = sheet
tail = (phase mod 2) xor sheet xor parity.
```

A **continuous carrier trace** is a list of states where each adjacent pair is related by one legal `roll` step for the corresponding input bit.

Main Claim

Theorem 5.1, Rolling Path Carrier. For every finite binary input stream, the rolling carrier produces a continuous trace of legal adjacent states. The trace preserves input order, maintains a binary head/tail dyad at every state, and can carry Paper 04 constraints as receipt payloads without treating the path as a straight-line jump.

Proof

The step rule is total on the finite state space:

```
{0,1} x {0,1,2,3} x {0,1}.
```

For every valid input bit, `sheet` changes by exactly one modulo 2, `phase` changes by exactly one modulo 4, and `parity` changes by XOR with the input bit. Therefore each step has a unique legal successor. A trace generated by successive applications of `roll` is continuous by construction because every adjacent pair is one legal step apart.

The head/tail dyad is a deterministic function of the current state, so it is defined at every position in the trace. A Paper 04 constraint can be attached to a trace position as payload because the carrier state and input index are replayable. The payload does not alter the rolling step rule, so carrying it does not break continuity. QED.

What This Paper Does Not Prove

This paper does not prove:

```
1. That the Oloid carrier predicts Rule 30 future bits without enumeration.
2. That the physical Oloid geometry has been fully formalized here.
3. That every curved path is a valid carrier.
```

It proves a finite rolling-state carrier and records the stronger claims as obligations.

Hand Reconstruction

- Draw eight rolling positions: two sheets times four phases.
- Start at `(sheet=0, phase=0, parity=0)`.
- Read a binary input bit.
- Flip sheet, advance phase, and XOR parity with the bit.
- Record the new head/tail dyad.
- Attach any Paper 04 constraint as a payload at the current step.
- Repeat for the input stream.
- Reject any path that skips a phase, skips a sheet flip, or uses a nonbinary bit.

Code Reconstruction

The production verifier is:

```
production/formal-papers/CQE-paper-05/verify_oloid_path_carrier.py
```

It verifies:

```
1. The rolling step is total for valid binary input.
2. Every adjacent trace pair is a legal step.
3. Head/tail dyads are binary at every state.
4. Paper 04 constraints can be attached as payloads without changing the path.
5. Invalid bits and discontinuous jumps are rejected.
6. Prediction claims remain out of scope.
```

Validation and Hidden-Guess Layer

Useful hidden-guess prompts:

```
What changes on every legal rolling step?
Does a curved carrier imply prediction?
What makes a path discontinuous?
Can a constraint payload alter the path rule?
```

Expected answers:

```
sheet flips, phase advances, parity XORs the bit
no
skipped phase/sheet or invalid bit
no
```

Open Obligations

- Wire `verify_oloid_path_carrier` into `cqe_engine.formal`.
- Connect Paper 04 constraint payloads to a shared route ledger.
- Separate physical Oloid geometry claims from finite rolling-state claims.
- Keep Rule 30 prediction as an explicit open problem until a verifier proves

it.

Conclusion

Paper 05 proves a path-carrier property, not a prophecy. A rolling carrier can preserve continuity across a non-straight path, carry repaired constraints, and emit replayable dyads at each step. That is the kernel needed by later papers.